From Geometric Intuition to Real-world Deployment with Support Vector Machines

Long Nguyen Hoang

April 17, 2026

University of Information Technology - VNU-HCM
Faculty of Information Science and Engineering

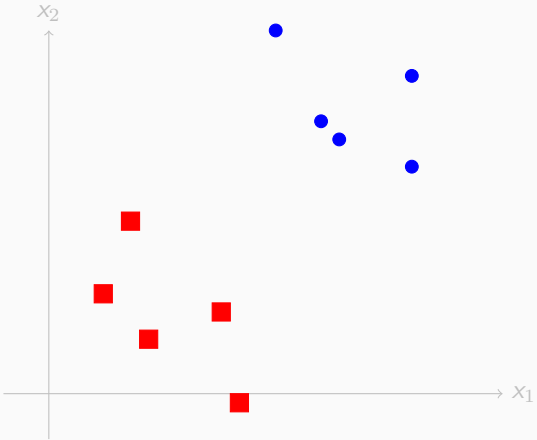
Nội dung bài chia sẻ

1. The Linear Intuition
2. Các mô hình SVM biên mềm (soft-margin SVMs)
3. Kernel Trick
4. Kỹ thuật triển khai SVM
5. Practice: Phân loại ảnh số viết tay 0 và 1

The Linear Intuition

Bài toán phân loại trên mặt phẳng

- Cho hai nhóm dữ liệu: Nhóm A và Nhóm O.
- Mục tiêu: Tìm một **đường ranh giới** để phân tách chúng.
- Có **vô số** cách vẽ đường thẳng.
- *Vậy đâu là ranh giới hợp lý nhất?*

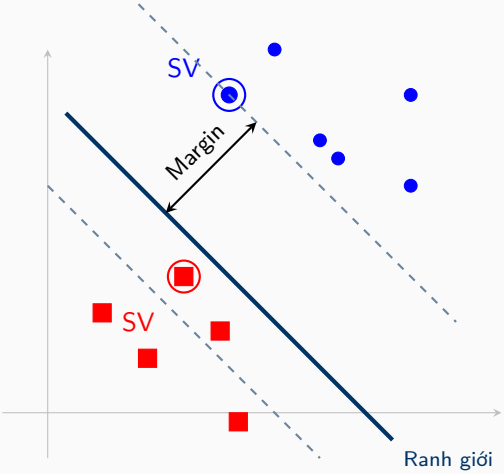


Ý tưởng cốt lõi: Đường ranh giới an toàn nhất

Làm sao để chọn đường tốt nhất?

1. Hãy tưởng tượng một **"con đường"** càng rộng càng tốt để phân tách hai nhóm.
2. **Đường ranh giới** sẽ nằm chính giữa con đường đó.
3. Những điểm nằm sát mép đường được gọi là **Support Vectors** (Điểm trụ cột).

Kết luận: Đường thẳng *an toàn nhất* là đường có **lề đường rộng nhất**.



Biểu diễn ranh giới trong không gian 2 chiều

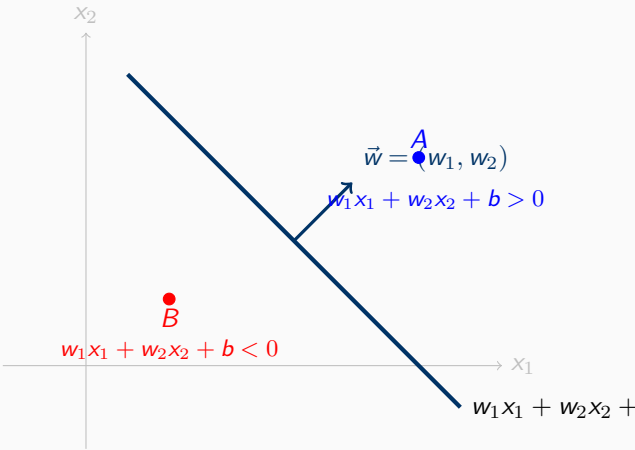
Một đường thẳng trên mặt phẳng $\mathbb{R}^2$ có dạng:

$$w_1 x_1 + w_2 x_2 + b = 0$$

Ý nghĩa các thành phần:

- $\vec{w} = (w_1, w_2)$: vector **vuông góc** với đường thẳng, quyết định *hướng*.
- b: điều chỉnh *vị trí* (tịnh tiến song song).

Khi thay tọa độ điểm $A(x_1^A, x_2^A)$ vào vế trái, dấu của kết quả cho biết điểm nằm *phía nào* của đường thẳng.



Tổng quát hóa: Siêu phẳng và bài toán

Từ 2D đến không gian d chiều

Trong không gian nhiều chiều, ranh giới là một **siêu phẳng**:

$$\boldsymbol{w}^T\boldsymbol{x} + b = 0$$

với $\boldsymbol{w} \in \mathbb{R}^d$, $b \in \mathbb{R}$.

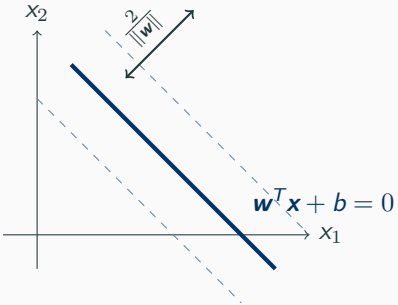
Khoảng cách từ điểm đến siêu phẳng:

$$d = \frac{|\boldsymbol{w}^T\boldsymbol{x} + b|}{\|\boldsymbol{w}\|}$$

Mục tiêu phân loại:

- Điểm lớp xanh (+1): $\boldsymbol{w}^T\boldsymbol{x} + b \geq +1$
- Điểm lớp đỏ (-1): $\boldsymbol{w}^T\boldsymbol{x} + b \leq -1$

$\Rightarrow y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1, \forall i$



Tổng quát mô hình

Giả định của mô hình:

- Dữ liệu huấn luyện $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$ với $\boldsymbol{x}_i \in \mathbb{R}^d$, nhãn $y_i \in \{-1, +1\}$.
- **Tuyến tính khả tách**: Tồn tại ít nhất một siêu phẳng phân tách hoàn toàn hai lớp.

Mô hình SVM (dạng hard-margin):

Siêu phẳng phân loại

$$f(\boldsymbol{x}) = \boldsymbol{w}^T \boldsymbol{x} + b$$

- Nếu $f(\boldsymbol{x}) \geq 0$: dự đoán lớp $+1$ (xanh)
- Nếu $f(\boldsymbol{x}) < 0$: dự đoán lớp -1 (đỏ)

Tổng quát mô hình

Điều kiện phân loại đúng với margin

Để đảm bảo mọi điểm nằm **ngoài hoặc trên mép lề**, ta yêu cầu:

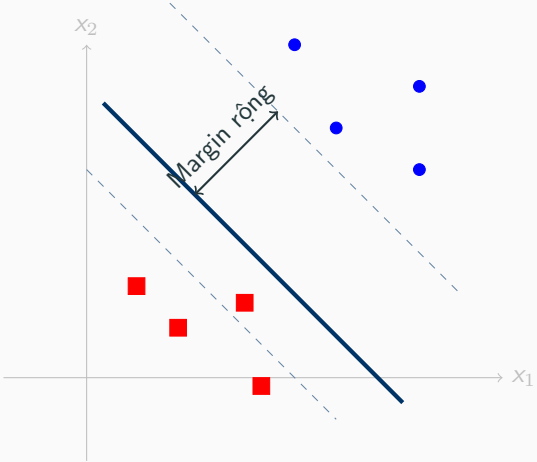
$$y_i \cdot (\boldsymbol{w}^T \boldsymbol{x}_i + b) \geq 1, \quad \forall i = 1, \ldots, n$$

Điều kiện sử dụng của SVM với biên cứng

Giả định lý tưởng:

- Hai lớp **hoàn toàn tách biệt**.
- Không có điểm nhiễu (outlier).
- Không có điểm bị gán nhãn sai.

Khi đó, tồn tại siêu phẳng phân tách với margin cực đại một cách **hoàn hảo**.

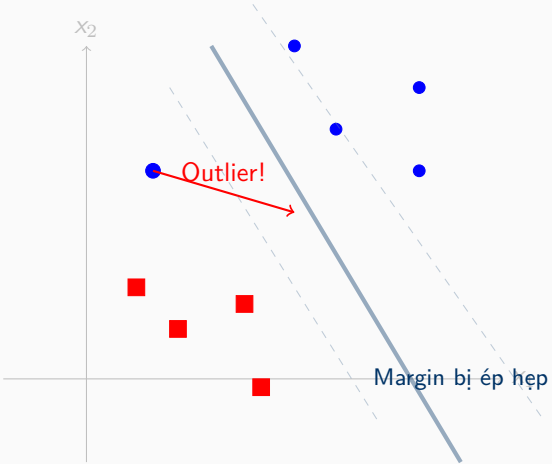


Điểm yếu của các mô hình Linear SVM

Trong thực tế:

- Dữ liệu thường bị **chồng lấn nhẹ**.
- Xuất hiện **outlier** (điểm nằm sai vùng).
- Có thể có **nhãn bị sai**.

Hậu quả: Không thể vẽ được một siêu phẳng vừa phân tách đúng *tất cả* điểm, vừa có margin hợp lý.



Điểm yếu của các mô hình Linear SVM

Ràng buộc cứng

$$y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1, \quad \forall i$$

- Với chỉ **một điểm outlier**, không tồn tại $\boldsymbol{w}, b$ nào thỏa mãn toàn bộ ràng buộc.
- Bài toán tối ưu trở thành **vô nghiệm** (infeasible).

Hai kịch bản tồi tệ

1. **Ép buộc phải phân tách đúng outlier**: Margin cực hẹp $\rightarrow$ mô hình overfitting, tổng quát hóa kém.
2. **Không tìm được nghiệm**: Thuật toán báo lỗi, không thể xây dựng mô hình.

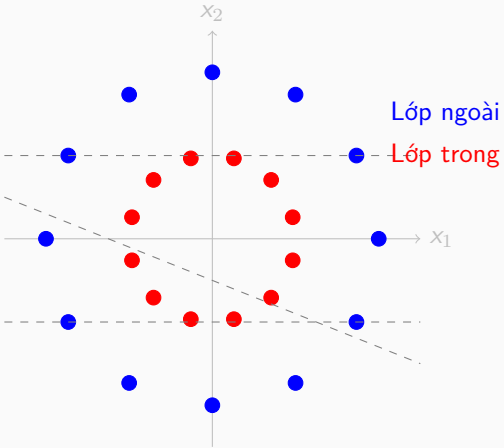
$\rightarrow$ **Cần một phiên bản SVM "mềm dẻo" hơn, chấp nhận sai sót có kiểm soát.**

Sự phi tuyến của dữ liệu

Ví dụ: Hai lớp tạo thành hai vòng tròn đồng tâm.

- Lớp trong (đỏ) bị bao bọc bởi lớp ngoài (xanh).
- **Không tồn tại** bất kỳ đường thẳng nào có thể phân tách chúng.
- Linear SVM **không thể áp dụng**.

Kết luận: Thế giới thực hiếm khi là tuyến tính. Cần công cụ mạnh hơn để xử lý các ranh giới phức tạp.



Vượt qua giới hạn của Hard-Margin

Hai vấn đề cốt lõi đã chỉ ra

1. **Dữ liệu nhiễu**: Hard-Margin sụp đổ trước outlier.
2. **Dữ liệu phi tuyến**: Không thể dùng đường thẳng phân tách.

Các hướng phát triển cần thiết:

1. Cho phép sai số có kiểm soát

- Thêm "khoan dung" cho mô hình.
- Chấp nhận một số điểm bị phân loại sai hoặc nằm trong margin.
- $\rightarrow$ **Soft-Margin SVM**

2. Biến đổi không gian

- Nâng dữ liệu lên không gian nhiều chiều hơn.
- Biến bài toán phi tuyến thành tuyến tính ở không gian mới.
- $\rightarrow$ **Kernel Trick**

Các mô hình SVM biên mềm (soft-margin SVMs)

Nhắc lại về điểm yếu của Linear SVMs

- Hard-Margin yêu cầu **mọi điểm** phải nằm ngoài margin và được phân loại đúng.
- Trong thực tế:
  - Dữ liệu thường có nhiễu, chồng lấn nhẹ.
  - Một điểm outlier có thể khiến bài toán **vô nghiệm**.
  - Hoặc margin bị ép rất hẹp $\rightarrow$ overfitting.

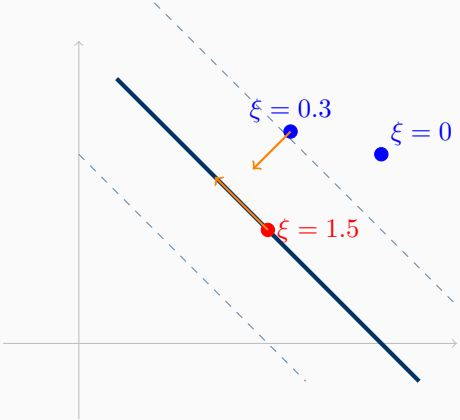
$\rightarrow$ **Cần một mô hình "mềm dẻo" hơn, cho phép một số điểm vi phạm margin.**

Biến lỏng slack ξ_i và điều kiện lề mềm (soft margin)

- **Biên cứng:** $y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1$ (quá cứng nhắc).
- **Biên mềm:** thêm $\xi_i \geq 0$ để nới lỏng:

$$y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1 - \xi_i$$

- ξ_i đo mức độ "vi phạm" của điểm i:
  - $\xi_i = 0$: điểm an toàn (ngoài margin).
  - $0 < \xi_i \leq 1$: lấn vào margin nhưng vẫn đúng lớp.
  - $\xi_i > 1$: bị phân loại sai.



Tại sao ràng buộc lại là $y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1 - \xi_i$?

Gốc rễ: Yêu cầu về khoảng cách trong hard-margin
Trong không gian, khoảng cách từ điểm $\boldsymbol{x}_i$ đến siêu phẳng là

$$\frac{|\boldsymbol{w}^T\boldsymbol{x}_i + b|}{\|\boldsymbol{w}\|}$$

.

Để đảm bảo độ rộng **margin** là $\frac{2}{\|\boldsymbol{w}\|}$, mọi điểm phải cách siêu phẳng **tối thiểu** $\frac{1}{\|\boldsymbol{w}\|}$.

Điều kiện đó được viết gọn thành: $y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1$.

Tại sao ràng buộc lại là $y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1 - \xi_i$?

Vậy $1 - \xi_i$ là gì?

- ξ_i là **khoảng cách vi phạm đã chuẩn hóa** (nhân với $\|\boldsymbol{w}\|$).
- Khi điểm $\boldsymbol{x}_i$ nằm quá gần hoặc sai phía, $y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b)$ sẽ nhỏ hơn 1.
- Lượng thiếu hụt đó chính là ξ_i:

$$\xi_i = \max(0,\, 1 - y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b))$$

- Do đó, yêu cầu $y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1$ trở thành $y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1 - \xi_i$.

Ví dụ trực quan

- Điểm A có $y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) = 0.7$. $\Rightarrow \xi_i = 1 - 0.7 = 0.3$ (Điểm lấn vào Margin).
- Điểm B có $y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) = -0.5$. $\Rightarrow \xi_i = 1 - (-0.5) = 1.5$ (Điểm bị phân loại Sai).

$\Rightarrow$ *ξ_i chính là câu trả lời cho câu hỏi: "Điểm này còn cách yêu cầu tối thiểu bao xa?"*

Thiết lập bài toán soft-margin SVM

Để mô hình hóa bài toán với dữ liệu nhiễu, ta kết hợp hai thành phần:

- **Thành phần hình học ($\mathbf{w}, b$):** Xác định siêu phẳng phân hoạch và độ rộng lề (Margin). Mục tiêu cốt lõi là tối đa hóa lề hình học $M = \frac{2}{\|\mathbf{w}\|}$.
- **Biến bù (slack variable ξ_i):** Đóng vai trò là ***sai số cho phép*** cho mỗi điểm dữ liệu $\mathbf{x}_i$:
  - $\xi_i = 0$: Điểm nằm đúng phía và ngoài *margin*.
  - $0 < \xi_i \leq 1$: Điểm nằm trong vùng *margin* nhưng vẫn đúng phía.
  - $\xi_i > 1$: Điểm bị phân loại sai (nằm sai phía siêu phẳng).

Tư duy tối ưu: Tìm sự cân bằng giữa việc *giữ lề rộng nhất có thể* và *giảm thiểu tổng mức độ vi phạm*.

Soft-Margin SVM: Hàm mục tiêu và Ràng buộc

Hàm mục tiêu

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \quad \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i$$

- $\frac{1}{2}\|\mathbf{w}\|^2$: margin rộng $\Leftrightarrow$ tổng quát hóa tốt.
- $\sum \xi_i$: tổng mức vi phạm margin (sai số).

Ràng buộc

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

Hinge Loss

$$\xi_i = \max(0,\, 1 - y_i(\mathbf{w}^T\mathbf{x}_i + b))$$

Tham số C: Lớn $\rightarrow$ ít sai, margin hẹp; Nhỏ $\rightarrow$ margin rộng, chấp nhận nhiễu.

Kernel Trick

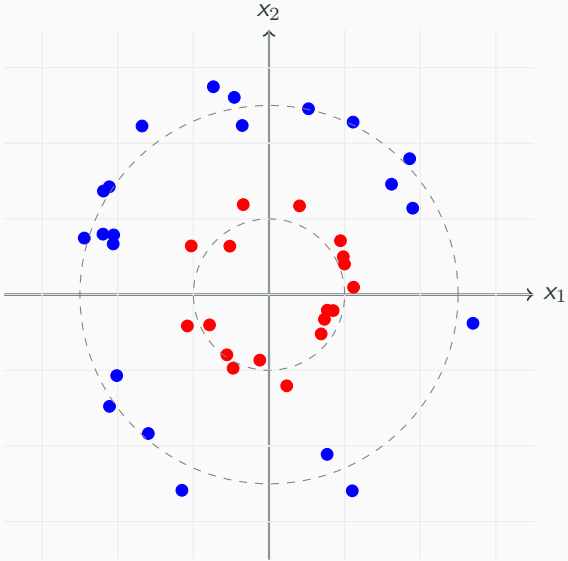
Hạn chế của SVM tuyến tính

- Cả Hard-Margin và Soft-Margin đều tìm **siêu phẳng tuyến tính**.
- Thực tế: nhiều tập dữ liệu có **ranh giới phi tuyến** (đường cong, vòng tròn).
- SVM tuyến tính thất bại hoàn toàn – dù có thêm slack ξ_i cũng không giải quyết được.

$\rightarrow$ **Cần một cách để mở rộng SVM sang các ranh giới phi tuyến.**

Ví dụ: Dữ liệu hai vòng tròn đồng tâm

- Lớp trong (đỏ) và lớp ngoài (xanh)
- **Không** đường thẳng nào phân tách được hai lớp; Soft-margin vẫn bất lực vì bản chất tuyến tính.



Ý tưởng: Ánh xạ đặc trưng $\phi(\mathbf{x})$

- Ánh xạ dữ liệu từ không gian gốc $\mathbb{R}^d$ lên không gian $\mathbb{R}^D$ ($D > d$, thường $D \gg d$).
- Trong không gian mới, dữ liệu có thể trở nên **tuyến tính khả tách**.

Ví dụ đơn giản

Với dữ liệu 2D: $X = (x_1, x_2) \in \mathbb{R}^2$, hai lớp đan xen.

Ánh xạ $X' = \phi(X) = (x_1, x_2, x_1^2 + x_2^2)$ lên 3D $\rightarrow$ dùng siêu phẳng phân tách dễ dàng.

- SVM trong không gian đặc trưng: thay $\mathbf{x}$ bằng $\phi(\mathbf{x})$.

Ví dụ: Ánh xạ hai vòng tròn đồng tâm

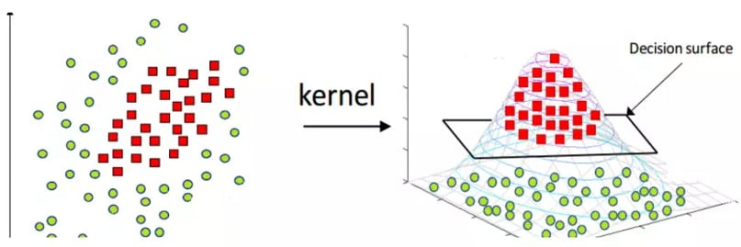


Figure 1: Dữ liệu gốc 2D (trái) và sau khi ánh xạ lên 3D (phải)

Không gian gốc 2D:

- Hai lớp tạo thành vòng tròn đồng tâm.
- Không đường thẳng nào phân tách được.
- Cần ánh xạ lên không gian cao hơn.

Sau khi thêm đặc trưng $z = x_1^2 + x_2^2$:

- Dữ liệu trải ra theo trục z (bán kính).
- Lớp trong có $z \approx 1$, lớp ngoài $z \approx 4$.
- Có thể dùng một siêu phẳng (ở đây là mặt phẳng) phân tách dễ dàng.

Tổng quát hóa ánh xạ đặc trưng $\phi(\mathbf{x})$

Tại sao cần nhiều đặc trưng phi tuyến?
Trong thực tế, ta không biết trước đặc trưng nào giúp tuyến tính hóa dữ liệu. Do đó, ta thường bổ sung một tập lớn các đặc trưng:

$$\phi(\mathbf{x}) = (1,\ x_1,\ x_2,\ x_1^2,\ x_2^2,\ x_1x_2,\ \dots)$$

- Với $\phi(\mathbf{x})$, ta hy vọng tìm được siêu phẳng trong không gian mới phân tách tốt dữ liệu.
- **Nhưng:** số chiều D của $\phi(\mathbf{x})$ có thể rất lớn (thậm chí vô hạn).

Thách thức của SVMs trong không gian đặc trưng

Bài toán gốc với $\phi(\mathbf{x})$

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^T\phi(\mathbf{x}_i) + b) \geq 1$$

- $\mathbf{w}$ có số chiều bằng với số chiều của $\phi(\mathbf{x})$, tức là D.
- Nếu D rất lớn, việc lưu trữ và tính toán với $\mathbf{w}$ trở nên **bất khả thi**.
- Tính $\phi(\mathbf{x}_i)$ cho mọi điểm cũng cực kỳ tốn kém.

Cần một cách tiếp cận khác

Làm sao để giải SVM mà không cần làm việc trực tiếp với vector $\mathbf{w}$ khổng lồ?

Bài toán đối ngẫu

Bài toán đối ngẫu (Dual Form)
Khi chuyển sang bài toán đối ngẫu, vector $\mathbf{w}$ bị triệt tiêu. Hàm mục tiêu mới **chỉ phụ thuộc vào tích vô hướng** giữa các điểm trong không gian đặc trưng:

$$\phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

- Vector trọng số được biểu diễn qua dữ liệu: $\mathbf{w} = \sum_i \alpha_i y_i \phi(\mathbf{x}_i)$.
- Hàm quyết định cũng chỉ cần tích vô hướng: $f(\mathbf{x}) = \sum_i \alpha_i y_i \phi(\mathbf{x}_i)^T \phi(\mathbf{x}) + b$.

Hệ quả then chốt
Nếu có cách tính $\phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$ **trực tiếp** từ $\mathbf{x}_i, \mathbf{x}_j$ mà không cần biết ϕ, ta sẽ giải quyết được bài toán trong không gian cao chiều một cách hiệu quả.

Vấn đề phát sinh với bài toán đối ngẫu

Vấn đề phát sinh

- Khi số chiều D của $\phi(\mathbf{x})$ rất lớn (thậm chí vô hạn), việc tính trực tiếp $\phi(\mathbf{x})$ (hay kể cả $\phi(\mathbf{x}_i)^T\phi(\mathbf{x}_j)$) trở nên **bất khả thi**.
- SVM cần tính rất nhiều tích vô hướng – nếu $D = 10^6$, mỗi phép nhân tốn $O(D)$.

Kernel Trick

Ý tưởng cốt lõi
Thay vì tính $\phi(\mathbf{x})$ rồi lấy tích vô hướng, ta định nghĩa một hàm **kernel** $K(\mathbf{x}_i, \mathbf{x}_j)$ sao cho:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

và tính **trực tiếp** từ $\mathbf{x}_i, \mathbf{x}_j$ trong không gian gốc.

Ví dụ cụ thể với $\phi(\mathbf{x}) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$

- Tích vô hướng trong không gian đặc trưng:
  $\phi(\mathbf{x})^T\phi(\mathbf{z}) = x_1^2z_1^2 + 2x_1x_2z_1z_2 + x_2^2z_2^2 = (\mathbf{x}^T\mathbf{z})^2$
- Vậy chỉ cần dùng **kernel đa thức bậc 2**: $K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^T\mathbf{z})^2$.

- Chi phí tính toán: $O(d)$ trong không gian gốc thay vì $O(D)$ với $D \gg d$ (thậm chí $D = \infty$).
- Kernel phải thỏa mãn **điều kiện Mercer** (đảm bảo tồn tại không gian đặc trưng tương ứng).

Một số kernel phổ biến

Linear Kernel

$$K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$$

- Tương đương SVM tuyến tính.
- Dùng khi dữ liệu gần tuyến tính.

Polynomial Kernel

$$K(\mathbf{x}_i, \mathbf{x}_j) = (\gamma \mathbf{x}_i^T \mathbf{x}_j + r)^d$$

- d: bậc đa thức.
- Phù hợp với ranh giới dạng đa thức.

RBF (Gaussian) Kernel

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2\right)$$

- Kernel được dùng **nhiều nhất**.
- Tương ứng không gian đặc trưng vô hạn chiều.
- $\gamma > 0$: tham số điều chỉnh độ phức tạp.

Ảnh hưởng của γ trong RBF Kernel

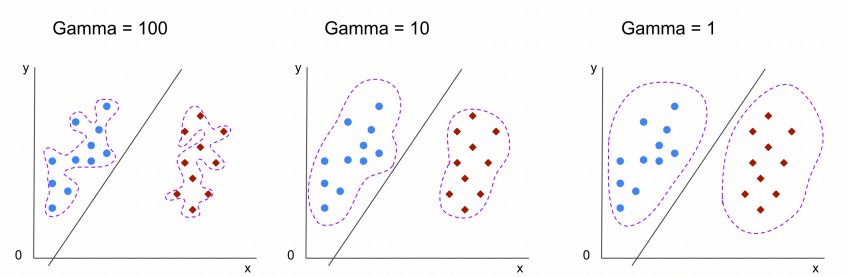


Figure 2: Minh họa ranh giới phân loại với γ lớn (trái) và γ nhỏ (phải)

- γ **nhỏ**: mỗi điểm ảnh hưởng rộng, ranh giới mượt, tổng quát hóa tốt.
- γ **lớn**: mỗi điểm ảnh hưởng hẹp, ranh giới uốn theo dữ liệu, dễ overfitting.

Tối ưu siêu tham số cho SVM với kernel

- SVM với kernel (đặc biệt là RBF) có hai siêu tham số quan trọng:
  1. C: cân bằng giữa độ rộng margin và mức phạt sai số.
  2. γ: kiểm soát độ phức tạp của ranh giới (độ uốn).
- Chọn thủ công khó đạt hiệu quả tối ưu.

Phương pháp phổ biến: Grid Search kết hợp Cross-Validation

- Thử nhiều cặp (C, γ) trên lưới giá trị (ví dụ: $C \in \{0.1, 1, 10, 100\}$, $\gamma \in \{0.001, 0.01, 0.1, 1\}$).
- Đánh giá bằng cross-validation, chọn cặp cho điểm tốt nhất.
- Trong thư viện `scikit-learn`: `GridSearchCV(SVC(), param_grid, cv=5)`.

Tổng kết: Kernel Trick

Vấn đề
SVM tuyến tính (cả hard và soft margin) thất bại với dữ liệu có ranh giới phi tuyến.

Giải pháp

- Ánh xạ dữ liệu lên không gian đặc trưng cao chiều $\phi(\mathbf{x})$ để dữ liệu trở nên tuyến tính khả tách.
- Nhờ **bài toán đối ngẫu**, mọi tính toán chỉ cần **tích vô hướng** $\phi(\mathbf{x}_i)^T\phi(\mathbf{x}_j)$.

Kernel Trick
Dùng **hàm kernel** $K(\mathbf{x}_i, \mathbf{x}_j)$ tính trực tiếp tích vô hướng mà không cần biết ϕ, cho phép làm việc với không gian vô hạn chiều (RBF) với chi phí tính toán thấp.

- Hai siêu tham số chính với RBF kernel: C (margin vs. sai số) và γ (độ uốn ranh giới).
- Thực tế: tối ưu (C, γ) bằng **grid-search** với **cross-validation**.

Kỹ thuật triển khai SVM

Quy trình huấn luyện SVM điển hình

1. **Thu thập và làm sạch dữ liệu**.
2. **Tiền xử lý:**
   - Chuẩn hóa đặc trưng (StandardScaler).
   - Xử lý giá trị thiếu, mã hóa biến phân loại.
3. **Chọn kernel và mô hình:**
   - Tuyến tính: `LinearSVC`.
   - Phi tuyến: `SVC(kernel='rbf')`.
4. **Tối ưu siêu tham số** (C, γ) với Grid Search + Cross-Validation.
5. **Huấn luyện và đánh giá** trên tập test.
6. **Triển khai** (lưu mô hình với `joblib` hoặc `pickle`).

Xử lý dữ liệu mất cân bằng

- SVM cố gắng tối đa margin chung, có thể bỏ qua lớp thiểu số nếu không điều chỉnh.
- Giải pháp đơn giản trong sklearn: `class_weight='balanced'`.
  - Tự động tăng trọng số phạt C cho các lớp ít mẫu.
- Cũng có thể truyền dictionary trọng số tùy chỉnh: `class_weight={0:1, 1:10}`.

Khi nào cần?

- Tỉ lệ lớp chênh lệch lớn (ví dụ 1:10, 1:100).
- Bài toán cần recall cao cho lớp thiểu số (phát hiện gian lận, bệnh hiếm).

Feature Scaling quan trọng như thế nào?

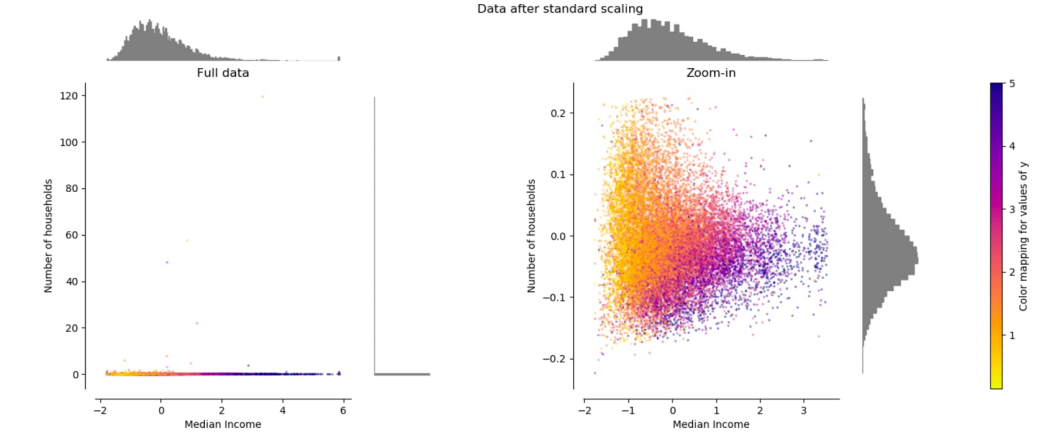
Tại sao SVM nhạy cảm với thang đo?

- SVM dựa trên khoảng cách (margin) và tích vô hướng.
- Nếu một đặc trưng có giá trị lớn hơn hẳn, nó sẽ chi phối hàm mục tiêu và làm méo margin.

Cách làm trong sklearn

- Dùng `StandardScaler` để đưa mỗi đặc trưng về trung bình 0, độ lệch chuẩn 1.
- **Luôn** fit scaler trên tập huấn luyện, sau đó transform cho cả tập kiểm tra.

Feature Scaling quan trọng như thế nào?



LinearSVC vs SVC

LinearSVC

- Chuyên cho kernel tuyến tính.
- Tối ưu bằng thuật toán coordinate descent (nhanh, ít tốn RAM).
- Hỗ trợ dữ liệu thưa (sparse).
- **Dùng cho dữ liệu lớn** ($> 10^5$ mẫu).

SVC(kernel='rbf')

- Linh hoạt, xử lý ranh giới phi tuyến.
- Độ phức tạp $O(n^2)$ đến $O(n^3)$ về bộ nhớ/thời gian.
- Không phù hợp nếu $n > 10^4$–10^5.
- Có thể dùng `RBFSampler` để xấp xỉ kernel.

Kinh nghiệm

- Bắt đầu với LinearSVC (baseline). Nếu underfit, thử RBF.
- Với RBF, giảm số mẫu hoặc dùng xấp xỉ nếu dữ liệu quá lớn.

Tối ưu C và γ với GridSearchCV

Code mẫu

```
from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVC
param_grid = {
'C': [0.1, 1, 10, 100],
'gamma': [0.001, 0.01, 0.1, 1]
}
grid = GridSearchCV(SVC(kernel='rbf'), param_grid, cv=5)
grid.fit(X_train_scaled, y_train)
print(grid.best_params_)
```

- Luôn thực hiện cross-validation (ví dụ cv=5) để đánh giá khách quan.
- Có thể dùng RandomizedSearchCV nếu không gian tham số quá lớn.
- Thời gian tìm kiếm tăng nhanh với số lượng mẫu – cân nhắc giảm lưới hoặc dùng tập con.

Lấy xác suất dự đoán từ SVM

- Mặc định SVM chỉ trả về nhãn cứng (± 1) hoặc khoảng cách đến siêu phẳng.
- Để có xác suất (probability), cần bật `probability=True` khi khởi tạo SVC.
- Cơ chế: Platt scaling – hồi quy logistic trên output của SVM (tốn thêm thời gian huấn luyện).

Lưu ý

- Xác suất thu được có thể không được hiệu chuẩn tốt (calibrated) – cân nhắc dùng `CalibratedClassifierCV`.
- Không nên dùng probability nếu chỉ cần nhãn, vì làm chậm đáng kể.

SVM với bài toán nhiều lớp (Multiclass)

- SVM nguyên bản chỉ phân loại nhị phân.
- Để xử lý K lớp ($K > 2$), có hai chiến lược chính:
  1. **One-vs-Rest (OvR)**: huấn luyện K bộ SVM, mỗi bộ phân biệt một lớp với phần còn lại.
  2. **One-vs-One (OvO)**: huấn luyện $\binom{K}{2}$ bộ SVM, mỗi cặp lớp một bộ.

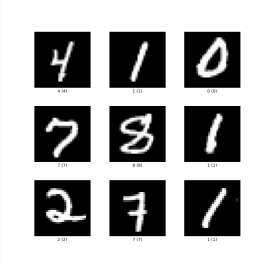
Trong sklearn

- `SVC` mặc định dùng **OvO** (phù hợp khi số lớp ít).
- `LinearSVC` mặc định dùng **OvR** (nhanh hơn cho nhiều lớp).

Practice: Phân loại ảnh số viết tay 0 và 1

Bài toán: Phân biệt chữ số 0 và 1

- **Mục tiêu:** Xây dựng mô hình SVM phân loại ảnh chứa chữ số viết tay **0** và **1**.
- **Dữ liệu:** Sử dụng tập MNIST (hoặc phiên bản thu gọn) – ảnh xám kích thước 28×28 pixels.
- **Vì sao chọn SVM?**
  - Dữ liệu nhỏ, số chiều trung bình (784).
  - Ranh giới giữa 0 và 1 gần như tuyến tính khả tách.
  - SVM hoạt động rất tốt trên bài toán này.



Tải dữ liệu và tiền xử lý

Code tải và lọc dữ liệu

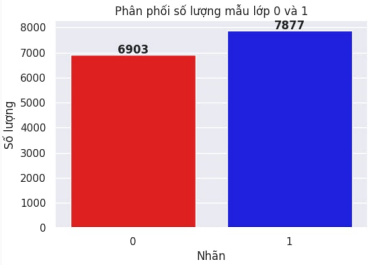
```
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split

mnist = fetch_openml('mnist_784', version=1)
X, y = mnist.data, mnist.target.astype(int)
mask = (y == 0) | (y == 1)
X_bin, y_bin = X[mask], y[mask]
```

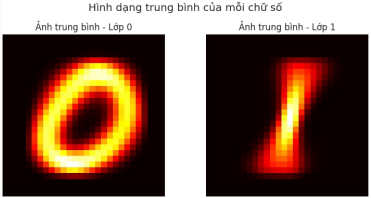
- **Phẳng hóa:** Ảnh 28×28 được duỗi thành vector 784 chiều.
- **Chuẩn hóa:** Dùng `StandardScaler` để đưa dữ liệu về phân phối chuẩn.

Lưu ý
Với SVM, **luôn chuẩn hóa** dữ liệu trước khi huấn luyện!

EDA: Phân phối nhãn và ảnh trung bình



- Dữ liệu khá cân bằng giữa hai lớp 0 và 1.
- Không cần áp dụng kỹ thuật cân bằng lớp.

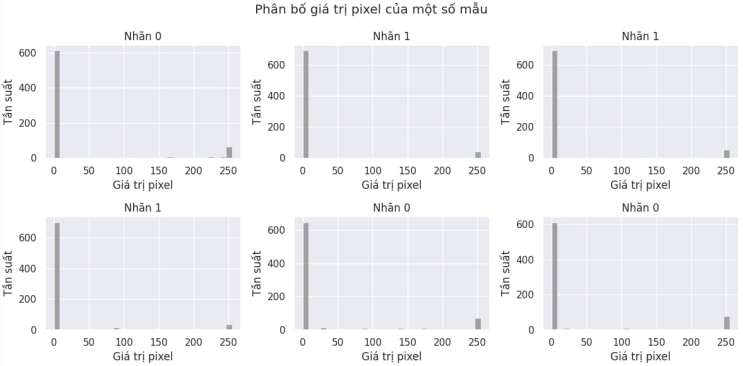


- Ảnh trung bình cho thấy sự khác biệt rõ rệt về hình dạng.
- Lớp 0 có xu hướng tròn, lớp 1 có nét thẳng đứng.

Nhận xét

Hai lớp có đặc trưng trực quan khá khác biệt, gợi ý rằng một mô hình tuyến tính có thể phân tách tốt.

EDA: Phân bố cường độ pixel



- Biểu đồ histogram cho thấy phần lớn pixel có giá trị thấp (nền tối).
- Các pixel sáng tập trung ở vùng chữ viết, tạo nên đặc trưng phân biệt.
- **Chuẩn hóa dữ liệu** (StandardScaler) là cần thiết để SVM không bị ảnh hưởng bởi thang đo.

Huấn luyện SVM tuyến tính (LinearSVC)

Pipeline đầy đủ

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('svm', LinearSVC(C=1.0, max_iter=10000))
])
pipeline.fit(X_train, y_train)
accuracy = pipeline.score(X_test, y_test)
print(f'Accuracy: accuracy:.4f')
```

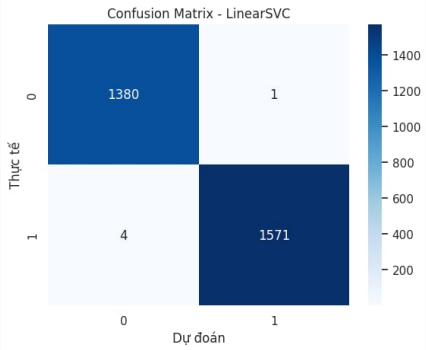
Huấn luyện SVM tuyến tính (LinearSVC)

- **Tham số** C**:** Giữ mặc định 1.0 (có thể tinh chỉnh sau).
- **max_iter:** Tăng lên để đảm bảo hội tụ.

Đánh giá mô hình

Confusion Matrix

- True Positive (1 được đoán đúng)
- True Negative (0 được đoán đúng)
- False Positive / Negative



Các metric

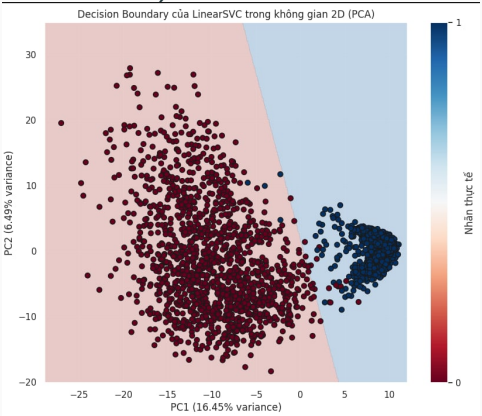
- Accuracy: $\frac{TP+TN}{Total}$
- Precision: $\frac{TP}{TP+FP}$
- Recall: $\frac{TP}{TP+FN}$
- F1-score

Kết quả điển hình
Với 0 vs 1, accuracy thường đạt $>$**99.5%**.

Trực quan hóa Support Vectors

- Mỗi điểm dữ liệu có hệ số α_i. Nếu $\alpha_i > 0$, điểm đó là **support vector**.
- Support vectors nằm trên hoặc trong lề margin, quyết định vị trí siêu phẳng.
- Đối với LinearSVC, có thể truy cập thuộc tính `svm.coef_` và `svm.intercept_`.

Minh họa (sau khi giảm chiều về 2D)



Thử nghiệm với Kernel RBF

Code so sánh

```
from sklearn.svm import SVC

pipeline_rbf = Pipeline([
    ('scaler', StandardScaler()),
    ('svm', SVC(kernel='rbf', C=1.0, gamma='scale'))
])
pipeline_rbf.fit(X_train, y_train)
print(pipeline_rbf.score(X_test, y_test))
```

- Với bài toán 0 vs 1, RBF không cải thiện đáng kể (vẫn 99.5%).
- **Thời gian huấn luyện** chậm hơn đáng kể so với LinearSVC.
- Kết luận: Chỉ dùng kernel phi tuyến khi thực sự cần thiết.

Tổng kết: SVM cho bài toán ảnh

- **Quy trình chuẩn:**
  1. Flatten ảnh $\rightarrow$ vector.
  2. Chuẩn hóa (StandardScaler).
  3. Chọn mô hình: LinearSVC cho nhanh, SVC+RBF nếu cần phi tuyến.
  4. Đánh giá và tinh chỉnh.
- **Khi nào dùng SVM cho ảnh?**
  - Dữ liệu nhỏ ($n < 10^4$).
  - Đặc trưng đã được trích xuất tốt (HOG, embeddings).
  - Cần mô hình giải thích được (support vectors).
- **Hạn chế:**
  - Không tận dụng được cấu trúc không gian 2D của ảnh.
  - Khó mở rộng với ảnh lớn, nhiều lớp.

$\rightarrow$ **Với ảnh phức tạp, Deep Learning (CNN) là lựa chọn tốt hơn.**

Q&A – Thực hành SVM

Arigathanksssss